İstanbul Bilgi University

EK-2 / APPENDIX-2
STAJ RAPORU KAPAK SAYFASI
INTERNSHIP REPORT COVER PAGE

ÖĞRENCİNİN / *STUDENT;*

ADI SOYADI *NAME SURNAME*	Oğuzhan Yıldırtan
NUMARASI *STUDENT NUMBER*	123200124
BÖLÜMÜ *DEPARTMENT*	Computer Engineering
DERS KODU *COURSE CODE*	CMPE 200
TARİH: *DATE*	
İMZA: *SIGNATURE*	

STAJ YAPTIĞI KURUMUN / *INSTITUTION OF INTERNSHIP;*

ADI *NAME*	Vadi Kurumsal
ADRESİ *ADDRESS*	[Institution address]
STAJ YAPILAN BİRİM *DEPARTMENT OF INTERNSHIP*	[Department of internship]
STAJ SORUMLUSUNUN ADI *INTERNSHIP SUPERVISOR*	Mustafa Arı
İLETİŞİM BİLGİLERİ *CONTACT INFORMATION*	[Supervisor contact information]
TARİH: *DATE*	
İMZA: *SIGNATURE*	

STAJIN / *INTERNSHIP;*

BAŞLAMA / BİTİŞ TARİHLERİ (GÜN/AY/YIL) *START/END DATES (DAY/MONTH/YEAR)*	27/07/2026 - 21/08/2026

Raf Library Management Platform Internship Report

Student: Oğuzhan Yıldırtan | Student number: 123200124 | Course: CMPE 200

Institution: Vadi Kurumsal | Supervisor: Mustafa Arı

Internship period: 27 July 2026 to 21 August 2026

Question 1 Information about the institution

Name and address

I completed my internship at Vadi Kurumsal. The project described in this report is the Raf Library Management Platform, a web application for managing a library catalogue, members and circulation records. My internship supervisor was Mustafa Arı.

Full registered name and address: [Confirm the institution's full legal name and complete postal address].

Brief history

[Add the institution's founding year, main field of activity and a short account of its development, using information confirmed by the institution].

Parent organisation

[State the parent organisation, if any. Write that the institution has no parent organisation only after confirming this information].

Existing facilities

[Describe the institution's premises and the facilities available to the internship unit, including the actual workspace and computing resources used].

Number of employees

[Insert the confirmed number of employees at the time of the internship].

Question 2 Hardware and software used

Development environment

The project was developed and run in a local macOS environment using a terminal, a web browser and a local PostgreSQL server. The browser was used to inspect the application, while terminal commands started the development services and ran builds and tests. The hardware specification should be recorded as follows: [Computer model, processor, memory and any institution-provided equipment actually used].

Frontend and backend technologies

React and TypeScript were used for the web interface. Vite provided the frontend development server and production build process. CSS defined the layout, colours, tables and forms, and lucide-react supplied interface icons. TypeScript types described the data exchanged with the API, including books, members, loans and dashboard statistics.

NestJS organised the backend into modules, controllers and services. Controllers exposed HTTP endpoints, while services contained the member registration, catalogue and circulation logic. The class-validator and class-transformer libraries supported input validation and transformation. The PostgreSQL driver provided parameterised queries, connection pooling and transactions.

Database and assistant tools

PostgreSQL stored bibliographic records, physical copies, members and loans. Primary keys and foreign keys linked these records, and a partial unique index protected active loans at copy level. SQL queries supplied catalogue availability, active loans and overdue records.

The Model Context Protocol SDK was used to implement a local tool server and a backend client. Zod defined tool input schemas. The assistant used the OpenAI JavaScript SDK with Gemini's compatible API endpoint; Gemini remained an optional external dependency. Vitest provided the automated unit tests, and pnpm managed the packages and workspace commands. Git and a .gitignore file supported preparation of the source code for version control.

Question 3 Assigned work and project participation

Project purpose and responsibilities

My project work focused on a library management application for staff. The application brings member registration, catalogue browsing, lending, returns and overdue monitoring into one interface. A separate assistant allows questions about the catalogue and active loan records through predefined tools. The application was developed as a learning project using demonstration data.

The technical responsibilities covered the user interface, API endpoints, database queries, circulation rules and integration of the assistant. The institution's supervisor was Mustafa Arı. Details of the organisational unit and collaboration should be completed from the actual internship arrangement: [Name of the unit, other team members if applicable, and how work was reviewed by the supervisor].

System architecture

The workspace contains three applications: a React web frontend, a NestJS API and an MCP server. The frontend sends HTTP requests to the API, which queries PostgreSQL. For assistant requests, the API checks the question's scope, contacts Gemini and forwards requested tool calls to the MCP server over standard input and output. The MCP server executes predefined, read-only SQL queries and returns their results.

This separation gave each part a clear responsibility. The frontend manages forms and presentation, the API applies business rules, and the database maintains persistent records. The assistant integration is optional, so catalogue and circulation functions can remain available when an API key is not configured.

Member registration and catalogue

The member module records a name, a Turkish identity number, a date of birth and optional contact details. Validation checks the format and checksum of the identity number and rejects invalid input. This is an algorithmic check; it does not verify a person's identity against an official registry. List responses display a masked identity number.

The catalogue presents the title, author, category, shelf and availability of each physical copy. A copy can be added with a unique inventory code, shelf code and optional ISBN. The service reuses a matching bibliographic record where possible and creates the physical copy

in a database transaction. The form and service make it possible to enter two copies of the same title with different inventory codes.

Physical copies and circulation

The original model treated a title as the item being lent. It was revised to distinguish an intellectual work from a physical copy. Bibliographic information is stored in books, and the copy's inventory code and location are stored in book_copies. A loan refers to the particular copy, so two copies of one title can be lent independently.

Existing records were accommodated by creating legacy copy records and associating old loans with them. Lending checks that the copy and member exist, locks the selected record, checks for an active loan and inserts the new loan within a transaction. The partial unique index on active copy loans provides an additional database safeguard against lending the same copy twice.

The return endpoint updates an active loan with a return date. The copy then becomes available again because availability queries look for loans whose returned_at value is null. Active loan lists show the borrowing and due dates, and overdue queries compare due dates with the database's current date.

Dashboard and user interface

The dashboard displays the number of physical copies, registered members, active loans and available copies. It also lists overdue returns and provides shortcuts to common tasks. The sidebar connects the overview, member, circulation, catalogue and assistant screens.

The interface includes member and copy forms, catalogue search and an availability filter. After a successful transaction, data is refreshed so the tables and summary cards reflect the changed state. A visible loading error was added so failed requests can be distinguished from an empty catalogue. The date shown in the welcome area was changed from a fixed value to the current date.

Assistant integration

Five MCP tools support book search, book status, member loans, overdue loans and available books. Their names are search_books, get_book_status, get_member_loans, list_overdue_loans and list_available_books. Tool definitions restrict the database operations the assistant can request; the model is not given a free-form SQL execution tool.

A scope filter classifies questions as library-related, sensitive or outside the intended domain. Identity numbers, birth dates, telephone numbers, email addresses and postal addresses are omitted from tool responses. Member names and loan details can still appear in relevant tool results. This distinction is important when describing the integration: restricting certain fields does not make all assistant data anonymous.

The assistant supports reading records. Registration, lending and returns remain operations in the staff interface. The integration limits the number of model and tool iterations to prevent an indefinite conversation loop. Provider availability and a valid API configuration remain necessary for live generated answers.

Question 4 Problems encountered and solutions

Catalogue modelling

A title-level uniqueness rule prevented the original data model from representing multiple physical copies. Lending also referred to the title, which meant one borrowed copy could make the whole title appear unavailable. Introducing book_copies and associating loans with copy IDs addressed this modelling problem. Each inventory code now identifies an independently lendable item.

Duplicate lending and member names

Two requests can inspect the same copy before either request completes. A check performed only in the interface would not protect the database from this situation. The loan service uses a transaction and a row lock, and the database has a partial unique index that allows only one active loan for a copy.

A separate uniqueness rule on member names could reject two different people with the same name. The name constraint was removed, while the identity-number uniqueness rule was retained. This change separates a display attribute from the field used to detect duplicate member records.

Validation and dates

Validating the length of a name before removing surrounding spaces could allow an input consisting entirely of spaces. Input transformation was added before validation for the relevant text fields. The member name is trimmed and checked for a non-empty value, making the validation closer to the value actually stored.

The lending form initially allowed a due date of today even though the service required a later date. The form's minimum date was changed to tomorrow. The due-date service compares against the current date in Europe/Istanbul, and frontend date generation was changed from a UTC ISO conversion to a local calendar date. These changes improved consistency for the intended local environment.

Tool compatibility and build configuration

A book-status query used PostgreSQL's similarity function, which depends on the pg_trgm extension. The code caught an error and issued a fallback query when the extension was missing. The query was simplified to use ordinary ordering, avoiding an unnecessary failed query on installations without that extension.

The API TypeScript configuration included tests outside the declared source root, causing a standalone type check to fail. A separate test configuration was introduced with the appropriate root and no emitted output. The application build and test type check could then be run independently.

Verification and remaining limitations

The project includes 12 unit tests across two files. They cover the identity-number algorithm and masking, and the chat scope filter's library, sensitive-data and follow-up cases. The test run passed. Application build checks were also used during development to identify compilation and packaging problems.

These checks have a limited scope. They do not establish full integration-test coverage for lending, data migration or live Gemini calls. The scope filter is based on text matching and is not a complete security boundary. Automated database integration tests, stronger migration handling and broader validation would be appropriate next steps if the prototype were extended.

Question 5 Professional skills and learning outcomes

The project helped me connect interface behaviour with API logic and database state. For example, selecting a copy in the lending form affects a specific loan record and changes that copy's availability. Following this flow made it easier to understand why the same rule must be represented consistently in the frontend, backend and database.

I improved my understanding of relational modelling by separating bibliographic records from physical copies. I also learned how transactions, row locks and unique indexes work

together to protect an operation. Removing the uniqueness rule on member names showed how an apparently convenient database constraint can conflict with the real-world meaning of the data.

The validation and date changes strengthened my debugging skills. Instead of treating a form error as an isolated interface issue, I examined the value before transformation, the service's checks and the data type used for storage. Build and test configuration work also helped me understand the difference between compiling application code and checking test code.

The assistant integration introduced a practical use of structured model tool calls. I learned to distinguish model-generated text from facts obtained through a controlled database query, and to describe the limits of automated tests and sensitive-field restrictions accurately.

Question 6 General evaluation of the internship

The library project gave me a concrete way to apply web development and database concepts within the internship. Its main result was a working prototype with a staff interface, member records, copy-based circulation and an optional assistant integration. It also provided examples of how requirements can change an initial design, particularly when multiple physical copies must be supported.

The most useful part of the work was tracing errors across the complete application. A catalogue or lending problem could originate in a constraint, a query, a validation rule or the interface. Reviewing these layers together improved the consistency of the application and made the reasons behind each change clearer.

The project remains a demonstration rather than a production service. It would benefit from database integration tests, controlled migrations, better handling of larger catalogues and authentication if it were used by a real organisation. Further assistant work would require testing the provider integration and improving the scope filter.

My overall evaluation is that the project provided useful practice in designing, implementing and reviewing an application. The next learning objective is to verify complete workflows with repeatable integration tests and to document the installation process so another developer can reproduce the same result.